

```

import { chromium } from 'playwright';
import { CONFIG } from './config.js';
import { fileURLToPath } from 'url';
import { dirname, join } from 'path';

const __dirname = dirname(fileURLToPath(import.meta.url));
const STATE_PATH = join(__dirname, '..', 'data', 'auth-state.json');

/**
 * Abre navegador usando sessão persistida.
 * Na primeira execução, você loga manualmente e a sessão fica salva.
 */
export async function launchBrowser() {
  const browser = await chromium.launch({
    headless: CONFIG.aiwf.headless,
    slowMo: CONFIG.aiwf.headless ? 0 : 100,
  });

  let context;
  try {
    context = await browser.newContext({
      storageState: STATE_PATH,
      viewport: { width: 1366, height: 900 },
      userAgent: 'Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537
    });
  } catch {
    // Primeiro login - sem estado salvo
    context = await browser.newContext({
      viewport: { width: 1366, height: 900 },
    });
  }

  return { browser, context };
}

/**
 * Scapa lista de vagas novas no AI Wordfinder.
 * IMPORTANTE: os seletores abaixo são placeholders.
 * Você precisa rodar `npm run login` primeiro, inspecionar os seletores
 * reais da plataforma e ajustar aqui.
 */
export async function scrapeNewJobs() {
  const { browser, context } = await launchBrowser();
  const page = await context.newPage();

```

```

try {
  await page.goto(`${CONFIG.aiwf.url}/jobs`, { waitUntil: 'networkidle' });

  // Verifica se está logado
  const isLoggedIn = await page.locator('[data-testid="user-menu"]').count() >
    || await page.locator('text=Dashboard').count() > 0;

  if (!isLoggedIn) {
    throw new Error('Sessão expirada. Rode `npm run login` pra relogar.');
  }

  // Ajuste: selecionar "Posted in last 24h" ou similar
  // await page.click('text=Last 24 hours');
  await page.waitForTimeout(1500);

  const jobs = await page.evaluate(() => {
    // AJUSTE ESSES SELETORES baseado na estrutura real do AI Wordfinder
    const cards = document.querySelectorAll('[data-testid="job-card"], .job-car

    return Array.from(cards).map(card => {
      const titleEl = card.querySelector('h2, h3, [class*="title"]');
      const linkEl = card.querySelector('a[href*="/job"]');
      const descEl = card.querySelector('[class*="description"], p');
      const budgetEl = card.querySelector('[class*="budget"], [class*="price"]'
      const idAttr = card.getAttribute('data-job-id') || linkEl?.href?.match(/\

      return {
        id: idAttr || Math.random().toString(36).slice(2),
        url: linkEl?.href || '',
        title: titleEl?.innerText?.trim() || '',
        description: descEl?.innerText?.trim() || '',
        budgetRaw: budgetEl?.innerText?.trim() || '',
      };
    }).filter(j => j.title && j.url);
  });

  // Pega descrição completa de cada vaga
  const enriched = [];
  for (const job of jobs.slice(0, 30)) {
    try {
      await page.goto(job.url, { waitUntil: 'networkidle' });
      const fullDesc = await page.locator('[class*="description"], [class*="det
      const budget = parseBudget(job.budgetRaw);
      enriched.push({
        ...job,
        description: fullDesc.slice(0, 3000),

```

```

        ...budget,
    });
    await page.waitForTimeout(500);
} catch (err) {
    console.error(`Falha ao enriquecer ${job.id}:`, err.message);
}
}

return enriched;
} finally {
    await context.storageState({ path: STATE_PATH });
    await browser.close();
}
}

function parseBudget(raw) {
    if (!raw) return { budget_type: 'unknown', budget_min: null, budget_max: null }

    const lower = raw.toLowerCase();
    const nums = raw.match(/\$?\s*(\d+(?:,\d{3})*(?:\.\d+)?) /g)?.map(n => parseFloa

    if (lower.includes('hour') || lower.includes('/hr') || lower.includes('hourly'))
        return { budget_type: 'hourly', budget_min: nums[0] || null, budget_max: nums
    }
    return { budget_type: 'fixed', budget_min: nums[0] || null, budget_max: nums[1]
}
}

```